Software Architecture and Design Specification

Campus Ride-Share Matching

Version: 1.0

Authors: Tanishkaa , Khushi P , Keertika , Pavithra

Date: 27 October 2026

1. Introduction

1.1 Purpose

This document describes the proposed architecture and design of Campus Ride-Share Matching.

1.2 Scope

It covers student ride offers and requests, location-based candidate retrieval, route-compatibility matching, match presentation, and in-app messaging/notifications.

1.3 Audience

Developers, testers, project evaluators, and maintainers.

1.4 Definitions

Geospatial query, route compatibility, ride offer, ride request, match score, API, and notification.

2. Document Overview

2.1 How to use this document

Use this document alongside the SRS and STP. It describes major components, interactions, design choices, interfaces, risks, and security considerations.

2.2 Related Documents

Software Requirements Specification (SRS), Software Test Plan (STP), requirements traceability matrix, and project source code.

3. Architecture

3.1 Goals and Constraints

Goals: understandable modular design, useful route matching, secure access to student data, and maintainable implementation. Constraints: match quality depends on provided location/time data; the chosen framework, database, and map provider must align with team resources.

3.2 Stakeholders and Concerns

Students: relevant matches, privacy, and ease of use. Developers: modularity and testability. Testers: repeatable matching results. Evaluators: clear traceability from requirements to design and tests.

3.3 Component Diagram

Insert a UML component diagram showing: Student UI → Application/API layer → Authentication and Ride Management services; the application layer uses Geospatial Search and Matching Engine; these components access the Database; Messaging/Notification service handles in-app conversations and match updates. Add any actual framework/database names only after confirming the implementation.

3.4 Component Descriptions

Student UI: collects ride details and presents matches/messages. Authentication/Access Control: identifies users and checks permissions. Ride Management: creates, updates, cancels, and retrieves offers/requests. Geospatial Search: filters candidate posts by pickup/drop-off area. Matching Engine: scores and ranks candidates using route compatibility and configured constraints. Persistence Layer: stores users, posts, locations, match status, conversations, and messages.
Messaging/Notification: stores and displays in-app messages and relevant updates.

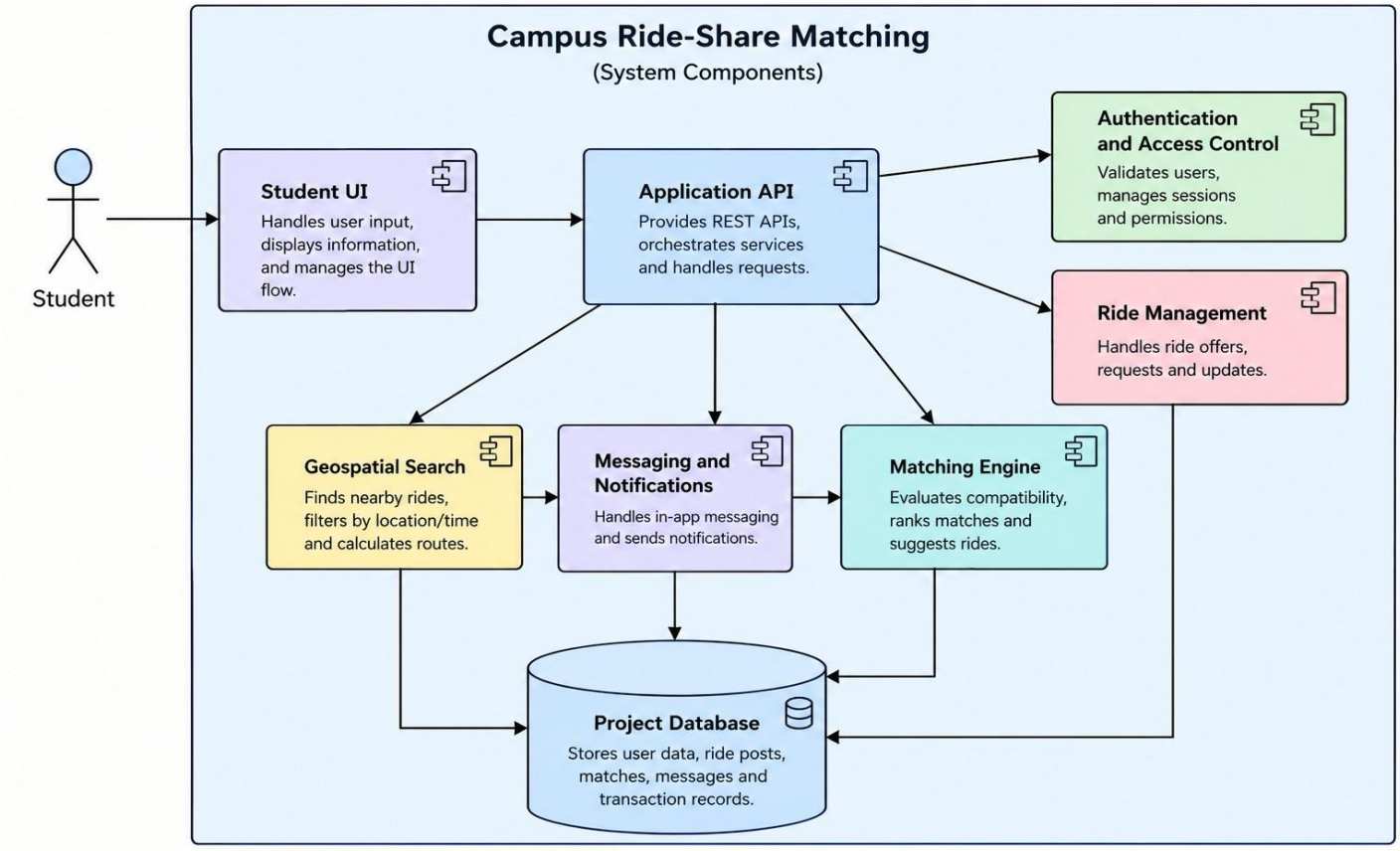
3.5 Architecture Pattern and Rationale

A modular layered architecture is suitable for a student project: presentation, application/business logic, and data access are separated. The matching logic is kept as a distinct module so its rules can be tested independently. A distributed microservices deployment is not required unless future scale or team constraints justify it.

3.6 Technology Stack and Data Stores

Use the actual stack selected by the team. The design requires a database that can store users, ride posts, coordinates/location fields, matches, and messages. If the selected database supports geospatial indexes/queries, use them for candidate retrieval; otherwise implement an explicitly documented alternative. Do not list technologies as implemented until verified in the repository.

Section 3.3 · Component Diagram



3.7 Risks and Mitigations

Inaccurate location data → validate and normalize location input. Poor match quality → document scoring rules and test known route scenarios. Unauthorized data access → enforce server-side ownership checks. External map-service failure → provide clear errors and use test fixtures for automated tests.

3.8 Traceability to Requirements

Ride Management supports FR-04 to FR-09. Geospatial Search supports FR-10 and FR-11. Matching Engine supports FR-12 to FR-15. Messaging/Notification supports FR-16 to FR-18. Authentication and Access Control support FR-01 to FR-03 and NFR-03.

3.9 Security Architecture

Require authentication for protected actions, enforce ownership checks on the server, validate user inputs, use HTTPS in deployment, keep secrets out of source control, and avoid logging passwords, tokens, or private message contents unnecessarily.

4. Design

4.1 Design Overview

The UI submits validated requests to the application layer. Ride Management stores offers and requests. Geospatial Search retrieves candidates, and the Matching Engine applies documented route and time constraints to rank them. The UI displays results and allows permitted users to communicate through in-app messaging.

4.2 UML Sequence Diagrams

Insert at least two sequence diagrams: (1) Student creates a ride offer/request: Student → UI → API/Access Control → Ride
Management → Database → UI confirmation. (2) Student searches and receives match suggestions: Student → UI → Search API → Geospatial Search → Matching Engine → Database/results → UI. Add a third diagram for messaging if required by the instructor.

4.3 API Design

Illustrative interfaces; adapt route names to the implementation. POST /api/rides/offers — create an offer; request includes pickup, destination, departure time, available seats and notes; response returns offer ID and status. POST /api/rides/requests — create a request; request includes pickup, destination, desired time and notes; response returns request ID and status. GET /api/matches?pickup=...&destination=...&time=... — retrieve ranked candidate matches. POST /api/conversations/{id}/messages — send an in-app message. Return clear validation, authentication, authorization, and server-error responses.

4.4 Error Handling, Logging, and Monitoring

Use consistent error responses for invalid input, missing posts, unauthorized actions, and unavailable services. Log operational events and errors without recording credentials or unnecessary private content. For the prototype, monitor failed requests, search/matching duration, and messaging errors if instrumentation is available.

4.5 UX Design

Use clear forms for offers and requests, readable location/time summaries, match scores or explanations where practical, explicit post status, and accessible validation messages. Confirm before destructive cancellation and show useful empty states when no compatible ride is found.

4.6 Open Issues and Next Steps

Confirm the final technology stack and data model; define the exact route compatibility formula and tie-breaking rules; decide whether a maps/geocoding provider is used; create UML component and sequence diagrams; and verify API paths against the implementation.

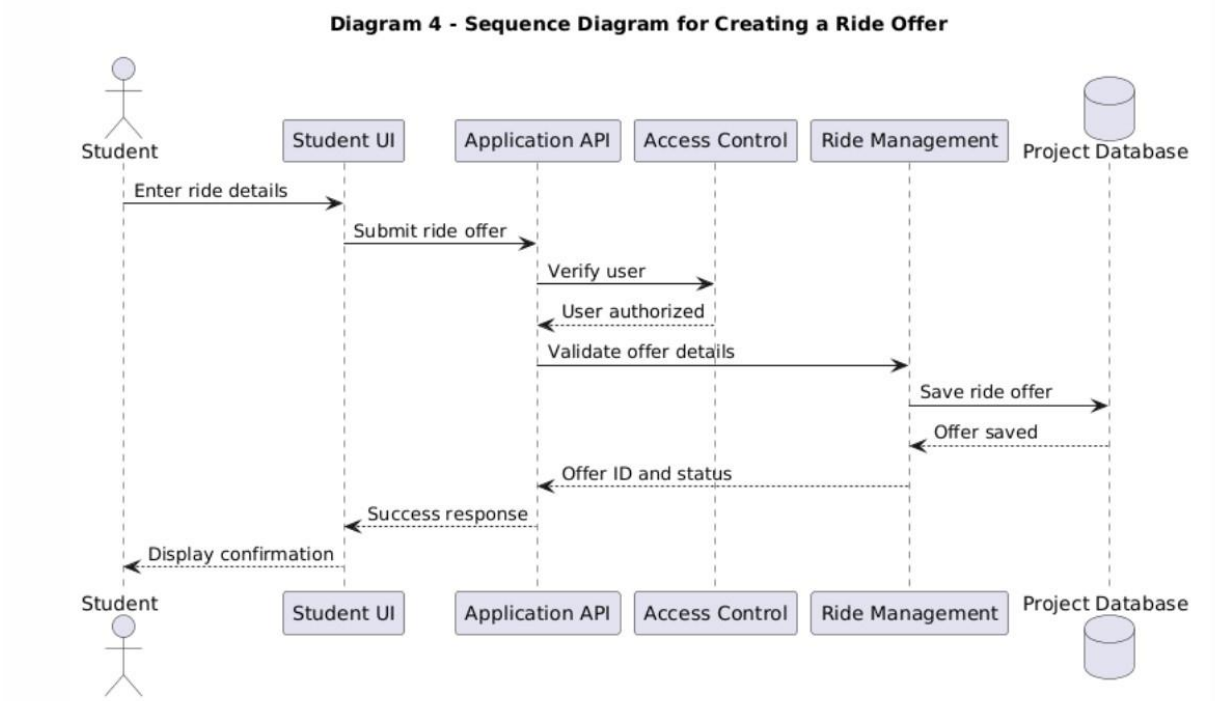
5. Appendices

5.1 Glossary

SRS: Software Requirements Specification. STP: Software Test Plan. SAD: Software Architecture and Design Specification.
API: Application Programming Interface. UI: User Interface.

5.2 References

Section 4.2 · Sequence Diagram 1



Section 4.2 · Sequence Diagram 2

